

Evaluation of Selected Algorithms for the Steiner Tree Problem

Abstract

The Steiner tree problem on graphs is a well-known NP-complete problem that extends the concept of spanning trees. Despite its complexity, various algorithms have been developed to solve it, both exactly and approximately. In this study, we implement and evaluate some of these algorithms, including an exact algorithm by Dreyfus and Wagner, an approximation algorithm by Kou, Markowsky and Berman, and a heuristic algorithm by Takashi and Matsuyama. We compare these algorithms in terms of running time and solution quality and evaluate their performance on a dataset of 200 graphs. Our results show that the exact algorithm is significantly slower than the approximation and heuristic algorithms, while the approximation and heuristic algorithms provide a good balance between running time and solution quality making them a good choice for most applications.

1 Introduction

The steiner tree problem is a well-known problem in graph theory. It is a generalization of the minimum spanning tree problem and is NP-hard.

As the problem is NP-hard, there is no known polynomial time algorithm that solves the problem exactly. However, there are several approximation algorithms that solve the problem with a guaranteed approximation factor as well as heuristic algorithms.

In this report, we will present three algorithms for the steiner tree problem and compare them in terms of running time and optimality. We will also present the results of our experiments and discuss the findings.

2 Preliminaries

2.1 Steiner Tree Problem

The problem is defined as follows: Given a undirected connected graph $G = (V, E, d)$ with a set of vertices V , a set of edges $E \subseteq V \times V$ and a function $d : E \rightarrow \mathbb{R}_+$ assigning a positive weight to each edge and a subset $T \subset V$, the goal is to find a tree S that spans all terminal vertices T and has minimal weight. The weight of a tree is the sum of the weights of its edges.

2.2 Optimal Decomposition Property

Dreyfus and Wagner [1972] introduced the optimal decomposition property of Steiner trees that is used in the exact algorithm in an dynamic programming approach. It is defined as follows.

Let S be any Steiner tree connecting T with $T \subset V$ is a subset of the vertices of a graph $G = (V, E)$ and let q be any node of T . If T contains at least 3 members then there exists a node $p \in V$ and a subset D of T such that:

D is a proper subset of $T \setminus \{q\}$ and $D \neq \emptyset$

S consists of 3 disjoint subsets S_1 , S_2 and S_3

S_1 connects $\{p, q\}$

S_2 connects $\{p\} \cup D$

S_3 connects $\{p\} \cup (T \setminus D \setminus \{q\})$

S_1 , S_2 and S_3 are all Steiner paths connecting their sets.

Dreyfus and Wagner [1972]

3 Algorithm & Implementation

3.1 Exact algorithm by Dreyfus and Wagner

An exact solution for the steiner tree problem can be found by using the Wagner-Dreyfus algorithm. The algorithm is based on the optimal decomposition property of steiner trees and uses the optimatimal decomposition property shown in section 2.2 in an dynamic programming algorithm. A drawback of the algorithm is that it calculates the size of the steiner tree but does not provide the tree itself. Dreyfus and Wagner showed that their algorithm has a time complexity of $|V|^3/2 + |V|^2(2^{|T|-1} - |T| - 1) + |V|(3^{|T|-1} - 2^{|T|} + 3)/2$ with $|V|$ being the number of vertices and $|T|$ being the number of terminal vertices. Dreyfus and Wagner [1972]

The algorithm is described in more detail in Algorithm 1.

3.2 Approximation algorithm by Kou, Markowsky and Berman

For the approximation algorithm, we use the well-known approximation algorithm by Kou et al. [1981] also known as KMB algorithm. The algorithm is based on the minimum spanning tree of the terminal vertices.

The algorithm works by first calculating the complete distance graph G_1 between all terminal vertices with the weight of the edges being the distance between the terminals. Then, the minimum spanning tree T_1 of G_1 is calculated. The spanning tree is then expanded into a graph G_s by replacing each edge with the shortest path in G between the vertices. Finally, the minimum spanning tree T_s of G_s is calculated. This tree can be pruned by removing edges so that every leave in T_s is a terminal vertex.

Kou et al. showed that the weight of T_s is at most $2(1 - \frac{1}{l})$ with l being the number of leaves in the minimal steiner tree.

In terms of complexity, they reported a running time of $\mathcal{O}(|T||V|^2)$ where $|V|$ is the number of vertices and $|T|$ the number of terminals.

Algorithm 1: Wagner-Dreyfus Algorithm

Data: Graph $G = (V, E, d)$, set of terminal vertices T

Result: Size of minimal steiner tree v

```

1 Choose  $q \in T$ 
2  $C \leftarrow T - \{q\}$ 
3 foreach  $t \in C$  do
4   foreach  $J \in V$  do
5      $S[\{t\}, J] \leftarrow d(t, J)$ 
6 for  $m = 2$  to  $|C| - 1$  do
7   foreach  $D$  such that  $D \subset C \wedge |D| = m$  do
8     foreach  $J \in V$  do
9        $u \leftarrow \infty$ 
10      foreach  $E$  such that
11         $D[1] \in E \wedge E \subset D$  do
12         $u \leftarrow \min(u, S[E, J] + S[D \setminus E, J])$ 
13      foreach  $I \in V$  do
14         $S[D, I] \leftarrow$ 
15           $\min(S[D, I], d(I, J) + u)$ 
16  $v \leftarrow \infty$  foreach  $J \in V$  do
17    $u \leftarrow \infty$ 
18   foreach  $E$  such that  $C[1] \in E \wedge E \subset C$  do
19      $u \leftarrow \min(u, S[E, J] + S[C \setminus E, J])$ 
20  $v \leftarrow \min(v, D(q, J) + u)$ 

```

The dominating part of the running time is the determination of shortest paths in the original graph for all edges in the minimum spanning tree T_1 . To improve the practical running time of the algorithm, we can parallelize this step trivially as there are no dependencies between the loop instances.

Algorithm 2: Kou, Markowsky and Berman algorithm (KMB 1981)

Data: Graph $G = (V, E, d)$, set of terminal vertices T

Result: Steiner tree $S \subseteq E$

```

1  $G_1 \leftarrow$  complete graph( $V_1, E_1, d_1$ ) with  $V_1 = T$ 
   and  $d_1(a, b) =$  distance ( $a \leftrightarrow b$ ) in  $G$ 
2  $T_1 \leftarrow$  minimum spanning tree of  $G_1$ 
3  $G_s \leftarrow$  graph with only vertices of  $G$ 
   /* This loop can be parallelized */
4 foreach  $e \in (T_1 \text{ edges})$  do
5    $P_{short} \leftarrow$ 
     shortest path in  $G$  between the vertices of  $e$ 
6   foreach  $e_{sp} \in P_{short}$  do
7      $\lfloor$  add  $e_{sp}$  to  $G_s$ 
8    $\rfloor$ 
9
10  $T_s \leftarrow$  minimum spanning tree of  $G_s$ 
11 while  $T_s$  has leaves that are not in  $T$  do
12    $\lfloor$  remove leaf from  $T_s$ 
13
```

3.3 Heuristic algorithm by Takashi and Matsuyama

Takahashi and Matsuyama [1979] proposed a simple greedy algorithm based on the shortest-path heuristic.

For that, it starts with a single (random) terminal vertex. Then, we iteratively add the closest terminal vertex to the current tree. This is done until all terminal vertices are included in the tree. The algorithm is described in more detail in Algorithm 3.

Takashi and Matsuyama showed that this algorithm has a running time of $\mathcal{O}(|T||V|^2)$.

Algorithm 3: Heuristic Algorithm

Data: Graph $G = (V, E, d)$, set of terminal vertices T

Result: Steiner tree $S \subseteq E$

```

1 choose  $V_S \leftarrow v \in T$ 
2 while  $V \setminus V_S \neq \emptyset$  do
3   choose  $v \in V \setminus V_S$  with minimal distance
     to a vertex  $v' \in V_S$ 
4    $P \leftarrow \{e \mid e \in \text{shortest path from } v \text{ to } v'\}$ 
5    $S \leftarrow S \cup P$ 
6    $V_S \leftarrow V_S \cup \{v'\}$ 
```

4 Experimental Evaluation

4.1 Evaluation Dataset

The algorithms were evaluated on the graphs from the "PACE 2018 - Steiner Tree" challenge. The graphs are available at PACE Challenge [2018].

The dataset features 200 different graphs with 52 to $\sim 20,000$ vertices and 80 to $\sim 200,000$ edges with a target set of terminal vertices with a size from 4 to 136. Further details regarding the dataset properties are described in 1.

Table 1: Dataset Graph Properties

	N	Mean	Std. Dev.	Min	Pctl. 25	Median	Pctl. 75	Max
Nodes	200	1,479.225	2,415.964	52	260.75	640	1,507.5	19,083
Edges	200	8,514.53	29,619.36	80	531.5	1,216	4,135	204,480
Terminals	200	19.435	17.72134	4	10	16	24	136

4.2 Data and Hardware

All algorithms were evaluated on a machine with an Apple M2 CPU with 3.49 GHz and a memory of 32GB. For any parallel parts of the algorithms, we used the OpenMP library with 8 threads. As the dataset contains quite large graphs and steiner tree instances, we used a timelimit of 600 seconds for the exact algorithm.

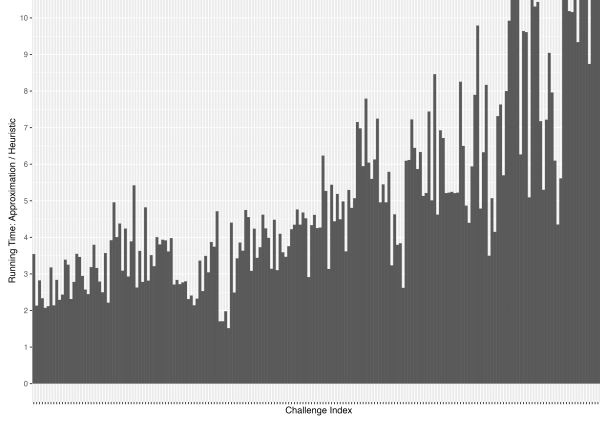


Figure 1: Running time comparison of the approximation and heuristic algorithm (instances sorted by $|V| * |T|^2$)

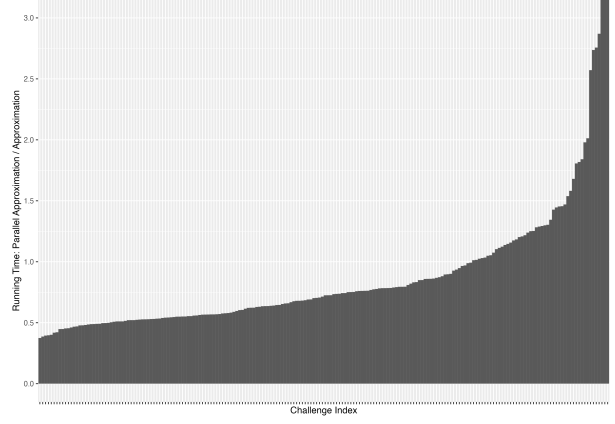


Figure 2: Speedup of the approximation algorithm with 8 threads (instances sorted by running time proportion)

4.3 Results

4.4 Running Time

Overview A general comparison between the running times of the algorithms is shown in the tables 2 and 3.

One can see that the exact algorithm (Dreyfus-Wagner) is significantly slower than the approximation and heuristic algorithm. On Average the algorithm takes more than 150.000 times longer than the heuristic algorithm and more than 50.000 times longer than the approximation algorithm.

Table 2: Running times of exact solvable instances in s

Variable	N	Mean	Std. Dev.	Min	Pctl. 25	Median	Pctl. 75	Max
Exact	46	135.3766	132.2473	0.004759	33.25303	99.8089	156.482	530.0443
Approximation	46	0.002610761	0.002128467	0.000175	0.000981	0.001643	0.00434325	0.006923
Parallel Approximation	46	0.002753174	0.001471633	0.001053	0.001719	0.002174	0.003498	0.007857
Heuristic	46	0.0008820435	0.000847613	0.000068	0.00033	0.000576	0.00111475	0.003669

Table 3: Running times of all instances in s

Variable	N	Mean	Std. Dev.	Min	Pctl. 25	Median	Pctl. 75	Max
Approximation	200	0.1483087	0.777834	0.000175	0.00370175	0.0094325	0.03320025	6.645585
Parallel Approximation	200	0.07149438	0.3523455	0.001053	0.0030635	0.0064475	0.0203305	3.021324
Heuristic	200	0.007845905	0.01782759	0.000068	0.00088175	0.0021245	0.0058345	0.122244

Runtime Benefits of Parallelization The parallelization of the approximation algorithm shows a speedup of up to 2.6 times as shown in Figure 2 but can also be slower as in 24% of the instances, the parallelized algorithm took at least as long as the non-parallelized alternative. Overall the parallelized algorithm took with a total running time over all instances of 14.3s only $\sim 48\%$ of the time of the non-parallelized algorithm and so was beneficial.

For a graph with as few as 500 nodes, the parallelized algorithm is already almost constantly faster than the non-parallelized algorithm. This is shown in Figure 12 in the appendix.

Comparison of Approximation and Heuristic Algorithm As shown in figure 1 the heuristic algorithm is faster than the approximation algorithm in 98% of the instances at least twice as fast and on average even ~ 6 times faster.

The speedup seems to be positively proportional to $|V| * |T|^2$.

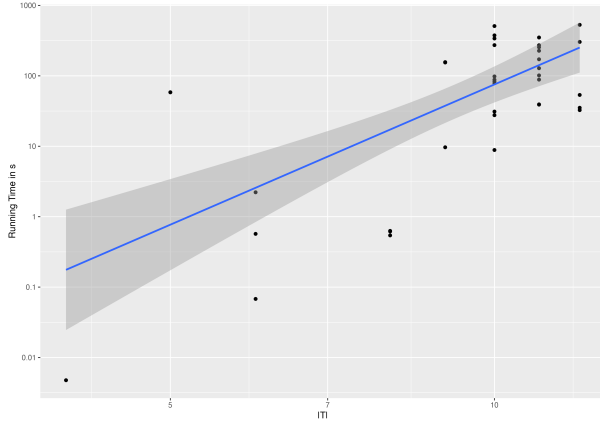


Figure 3: Exact: Relationship between theoretical complexity and practical running time

4.5 Theoretical Complexity and Practical Running Time

Dreyfus-Wagner Algorithm Wagner and Dreyfus reported a theoretical running time proportional to $\mathcal{O}(x^{|T|})$

We compare that to our practically observed running times by plotting the logarithm of the running time against the number of terminals in Figure 3.

As one can see, the plot shows a straight line which represents an exponential relationship between the number of terminals and the running time within a semi-log plot.

Kou, Markowsky and Berman Algorithm Kou et al. reported a complexity of $\mathcal{O}(|T||V|^2)$ for their algorithm.

We compare that to our practically observed running times by plotting the logarithm of the running time against the logarithm of $|T||V|^2$ in a log-log plot (Figure 4).

One can see a straight line in the plot which shows a relationship of the form $y = ax^b$ where b is the slope of the line. This indicates that the complexity also holds in practice.

In figure 5, the same plot is shown for the parallelized version of the algorithm. the line appears to

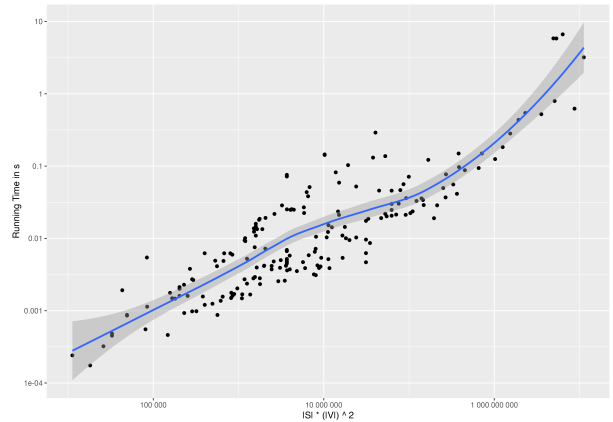


Figure 4: Approximation: Relationship between theoretical complexity and practical running time

be more convex, indicating that for small instances the parallelization does not pay off but for larger instances, the parallelization is beneficial.

Heuristic Algorithm Takahashi and Matsuyama reported a running time of $\mathcal{O}(|T||V|^2)$ for their algorithm.

We did the same plot (6) for the heuristic algorithm as well and see that the theoretical running is matching the practical running time as well.

4.6 Solution Quality

The found Steiner trees of the approximation and heuristic are both relatively close to the reported size of the minimal Steiner tree with a median error of $\sim 8\%$ and a mean error of $\sim 23\%$. The difference between the approximation and heuristic algorithm in terms of solution quality does not seem to be relevant as shown in table 4.

In the appendix, we show the solution quality of the approximation and heuristic algorithm in more detail in figures 10 and 11 and for every instance in figure 9.

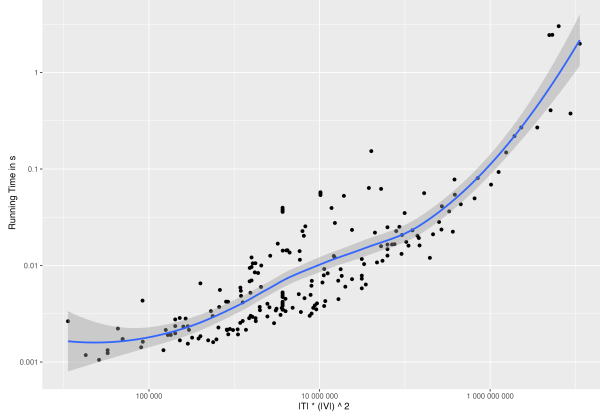


Figure 5: Approximation Parallel: Relationship between theoretical complexity and practical running time

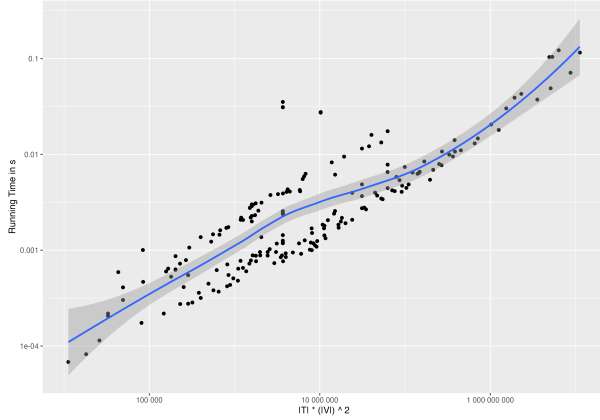


Figure 6: Heuristic: Relationship between theoretical complexity and practical running time

Table 4: Approximation and Heuristic Solution Quality

Variable	N	Mean	Std. Dev.	Min	Pctl. 25	Median	Pctl. 75	Max
Minimal Steiner tree size	200	496.079.6	1.009.989	15	560	1.588.5	9.915.25	3.900.450
Approximation Prop.	200	1.231885	0.267404	1	1.048483	1.084271	1.362538	1.92291
Heuristic Prop.	200	1.228787	0.2602824	1	1.049371	1.081373	1.362538	1.856864

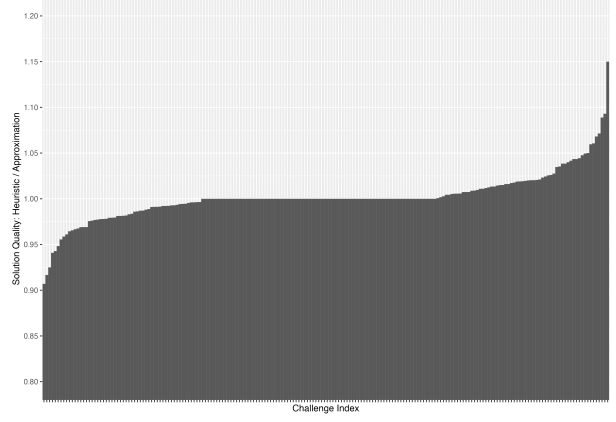


Figure 7: Solution quality of the approximation in comparison to the heuristic algorithm

5 Discussion and Conclusion

In conclusion, we have presented three algorithms for the Steiner tree problem: an exact algorithm by Dreyfus and Wagner, an approximation algorithm by Kou, Markowsky and Berman, and a heuristic algorithm by Takashi and Matsuyama.

We compared these algorithms in terms of running time and solution quality. The exact algorithm, while solving the problem optimally, is significantly slower than the approximation and heuristic algorithms. The approximation algorithm and heuristic algorithm provided a good balance between running time and solution quality, with a running time proportional to $|T||V|^2$.

In our experiments, we evaluated the algorithms on a dataset of 200 graphs and found that the heuristic algorithm were faster than the exact algorithm in practise while maintaining a very similar solution quality.

The parallelization of the approximation algorithm showed a speedup of up to 2.6 times, making the approximation algorithm more efficient, even for instances with just above 500 vertices.

In terms of solution quality, both the approximation and heuristic algorithms produced Steiner trees that were relatively close to the size of the minimal

Steiner tree. The median error was around 8%, indicating that these algorithms can provide good approximations to the optimal solution.

Overall, depending on the specific requirements of the problem, either the approximation algorithm or the heuristic algorithm can be suitable choices, offering a good balance between running time and solution quality. The exact algorithm is only suitable for small instances of the problem and if the exact solution is absolutely necessary.

References

- S. Dreyfus and R. Wagner. The steiner problem in graphs. *Networks*, (1):195–207, 1972.
- GeeksforGeeks. Steiner tree, 2023. URL <https://www.geeksforgeeks.org/steiner-tree/>.
- L. Kou, G. Markowsky, and L. Berman. A fast algorithm for steiner trees. *Acta Informatica*, (15): 141–145, 1981.
- PACE Challenge. Steiner tree: Track 1 - exact with few terminals, 2018. URL <https://pacechallenge.org/2018/steiner-tree>.
- H. Takahashi and A. Matsuyama. An approximate solution for the steiner problem in graphs. *Mathematica Japonica*, (24):573–577, 1979.

Appendix

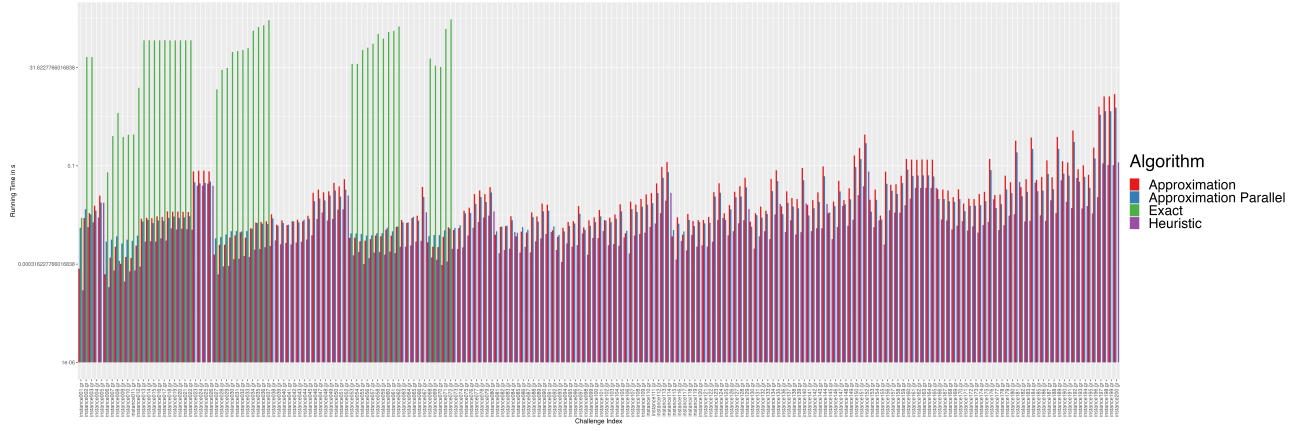


Figure 8: Running time of all algorithms on all instances

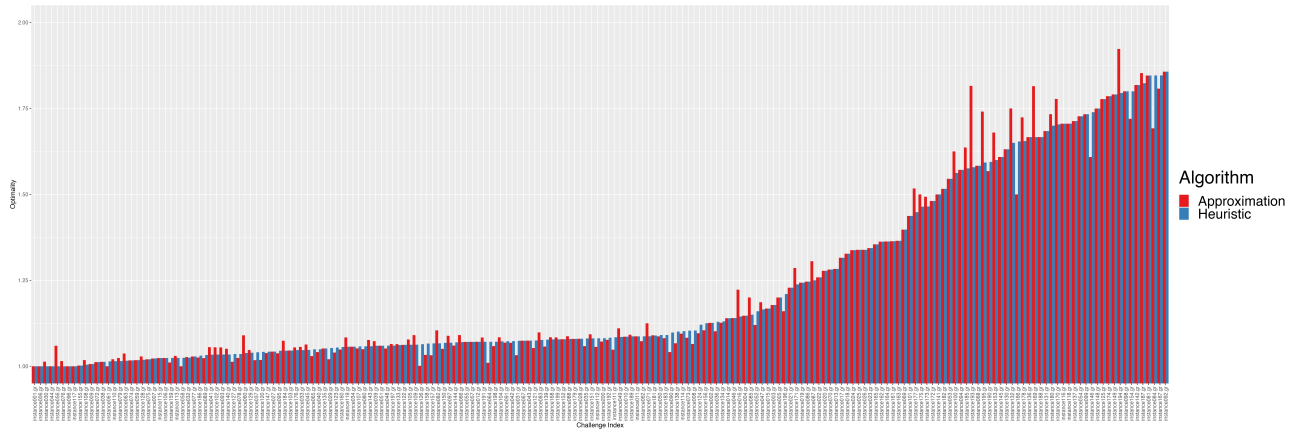


Figure 9: Solution quality of the approximation and heuristic algorithm

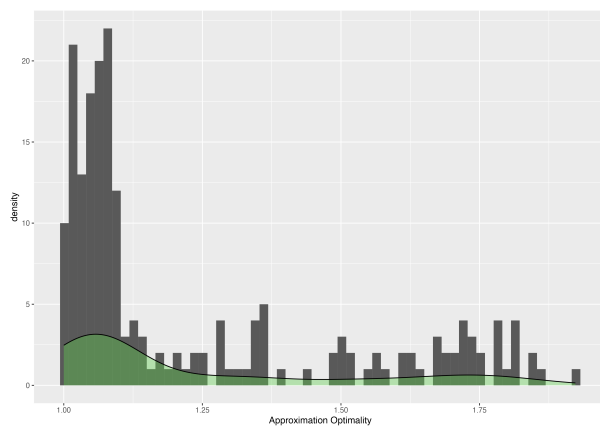


Figure 10: Solution quality of the approximation algorithm

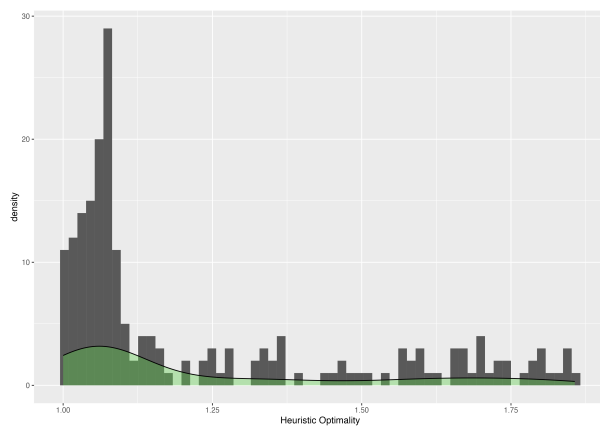


Figure 11: Solution quality of the heuristic algorithm

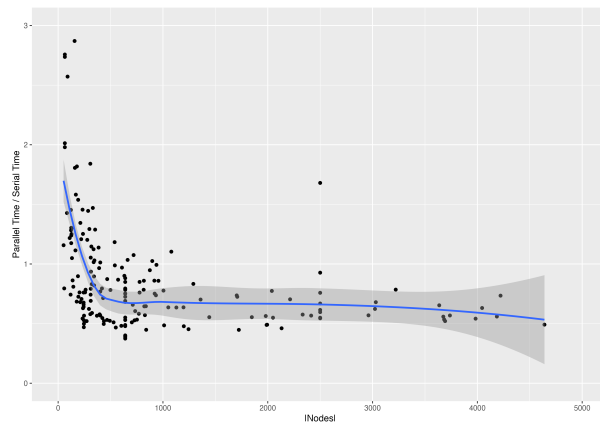


Figure 12: Break-even point of the parallelized approximation algorithm